Introduction to computingEngineering and the Build Environment FCE009

ProjectReport

Arduino based Temperatureandhumidity sensing hardware project along with mobile application

Submittedby:

Ali KhalilAliKhalil

Table of contents:

1. Introduction

2. Methodology

3. Results

4. Discussion and Reflection

5. References List

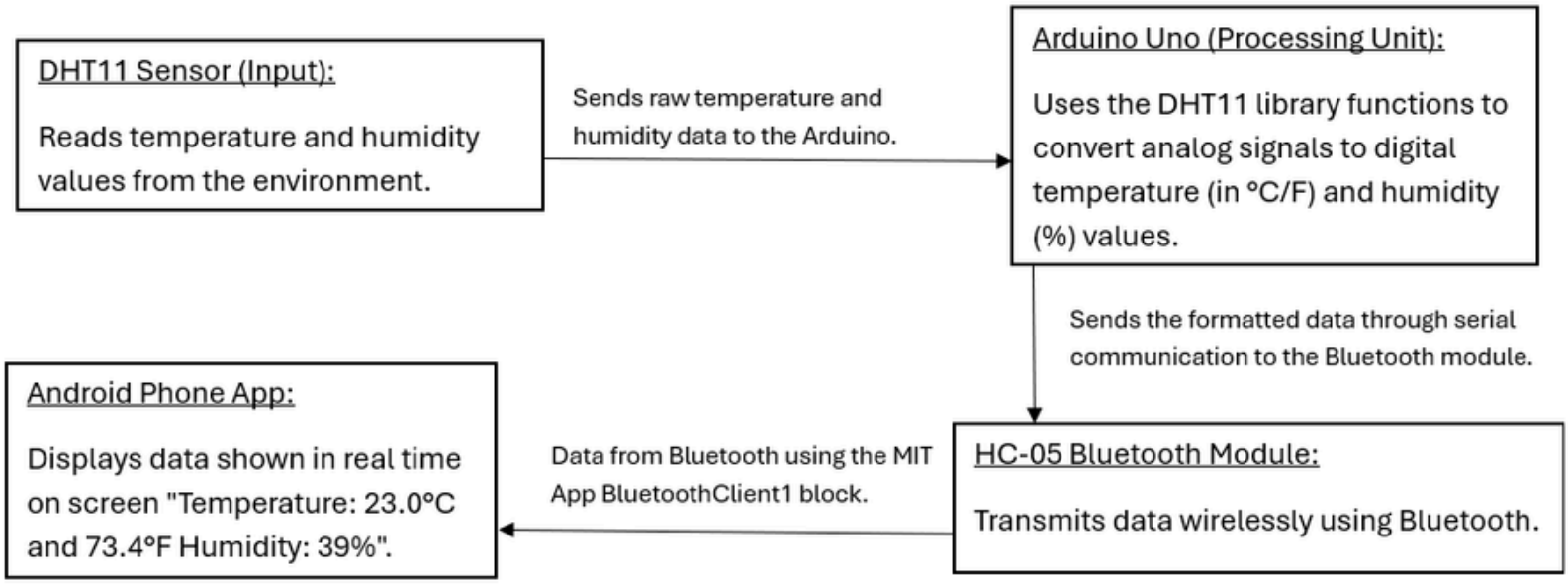
1. Introduction

This report describes how to build an Arduino based environmental monitoring system using the HC-05 Bluetooth module and the DHT11 temperature sensor. The main goal of this project is to collect environmental data in real time which is temperature and humidity and wirelessly send it to a phone to display the data. As Mori and Swaminarayan (2020) suggested, this system is an example of an Internet of Things (IoT) in which physical conditions are monitored and sent to a user interface for observation. This can be used in devices like air conditioner controllers that display the room's temperature and humidity (Srivastava et al., 2018).

The Arduino Uno microcontroller was chosen because of its compatibility, simple programming interface, and reliability. It receives sensor data and serves as a processing unit. The DHT11 sensor, that has a thermistor and a humidity sensing component, measures both temperature and humidity (Debele and Qian, 2020). The NTC thermistor inside the DHT11 operates by changing its resistance with temperature, although the DHT11 conducts the calculations internally, it uses the Beta equation:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} ln \left(\frac{R}{R_0} \right)$$ (ArduinoDIY, 2015)

In analog sensors, this formula is used to convert resistance into temperature. However, the DHT11 takes away this translation by sending digital signals directly to the Arduino. The Arduino then turns incoming sensor signals into useful data. After that, it uses the HC-05 Bluetooth module to communicate wirelessly. Bluetooth uses serial communication to transmit data between the Arduino and the Android phone through short range radio frequency (RF) signals (Mori and Swaminarayan, 2020). Once linked, the Android app receives and displays temperature and humidity data in real time.



2. Methodology

The first step in setting up the hardware was to connect the Arduino Uno board to a laptop by USB connection. This was required to power the Arduino, but also to upload code created in the Arduino Integrated Development Environment (IDE). It was necessary to ensure that the Arduino IDE was already installed on the laptop before beginning, as this software makes it possible for code editing and uploading to an Arduino board. Once the IDE was confirmed to be put in place, the Arduino was turned on and ready for programming.

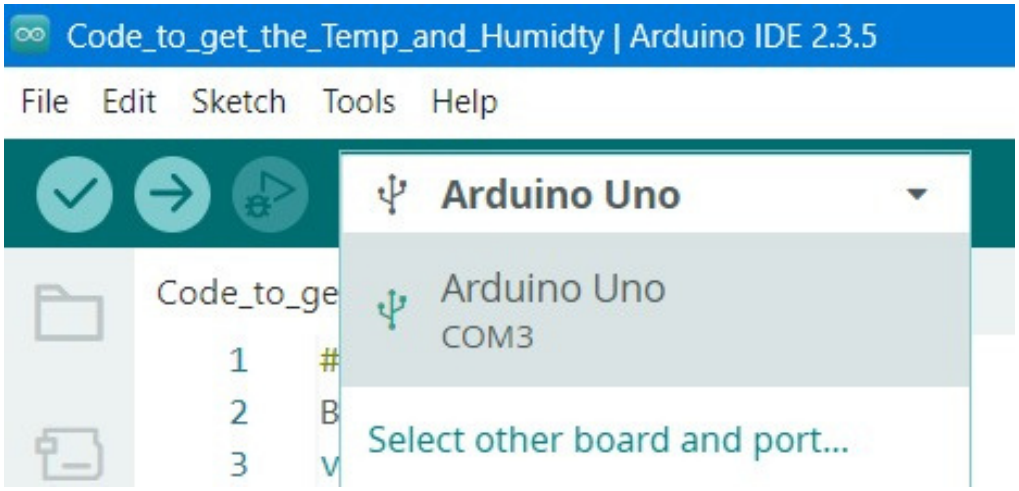


Figure 1: Arduino IDE downloaded and connected to correct Arduin Uno Microcontroller

After that, it was time to bring together the hardware on the breadboard. This included the Arduino Uno microcontroller, the DHT11 temperature and humidity sensor, and the HC-05 Bluetooth. The breadboard provided an organized section, making it easier to debug and adjust wires during development. According to Circuit Basics (n.d.), the DHT11 sensor was placed on the breadboard, with three important connections made using the jumper wires. The DHT11's VCC pin was connected to the Arduino's 5V pin to supply power. The data pin was then linked to the Arduino's digital pin 7, which would be used to read the sensors outputs. To finish the circuit, the GND pin was linked to the ground of the Arduino.

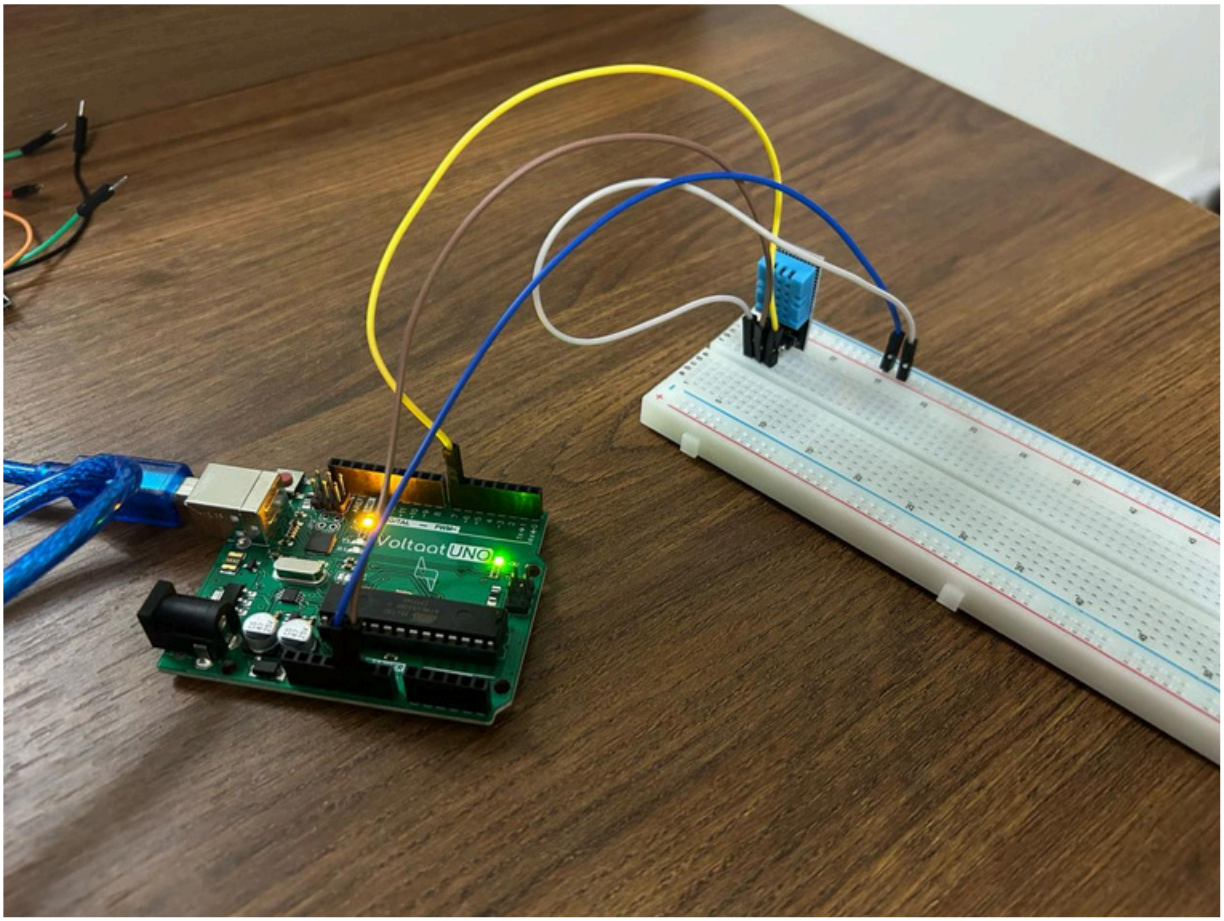


Figure 2: DHT11 sensor (Thermistor) connected onto Breadboard

After looking over the website Dejanm (n.d.), the next step was to take the HC-05 Bluetooth module and carefully place it into the breadboard, making sure that its pins were fully in the holes for a secure connection. The Arduino's RX (receive) pin was connected to the TX (transmit) pin of the Bluetooth, while the Arduino's TX pin, was connected to the HC-05's RX pin. This is important because it provides transmission of data in both directions where data provided from the Arduino is received by the Bluetooth module, and vice versa. In addition, the power supply connections were made to power the Bluetooth module. The 5V pin on the Arduino connected directly to the VCC pin on the Bluetooth providing the necessary power. Finally, to complete the power circuit, the Arduino's GND (ground) pin was linked to the Bluetooth GND pin. These stages set the foundation for wireless communication between the Arduino and the circuit broad with the mobile device, which was accomplished through programing the code and the MIT App.

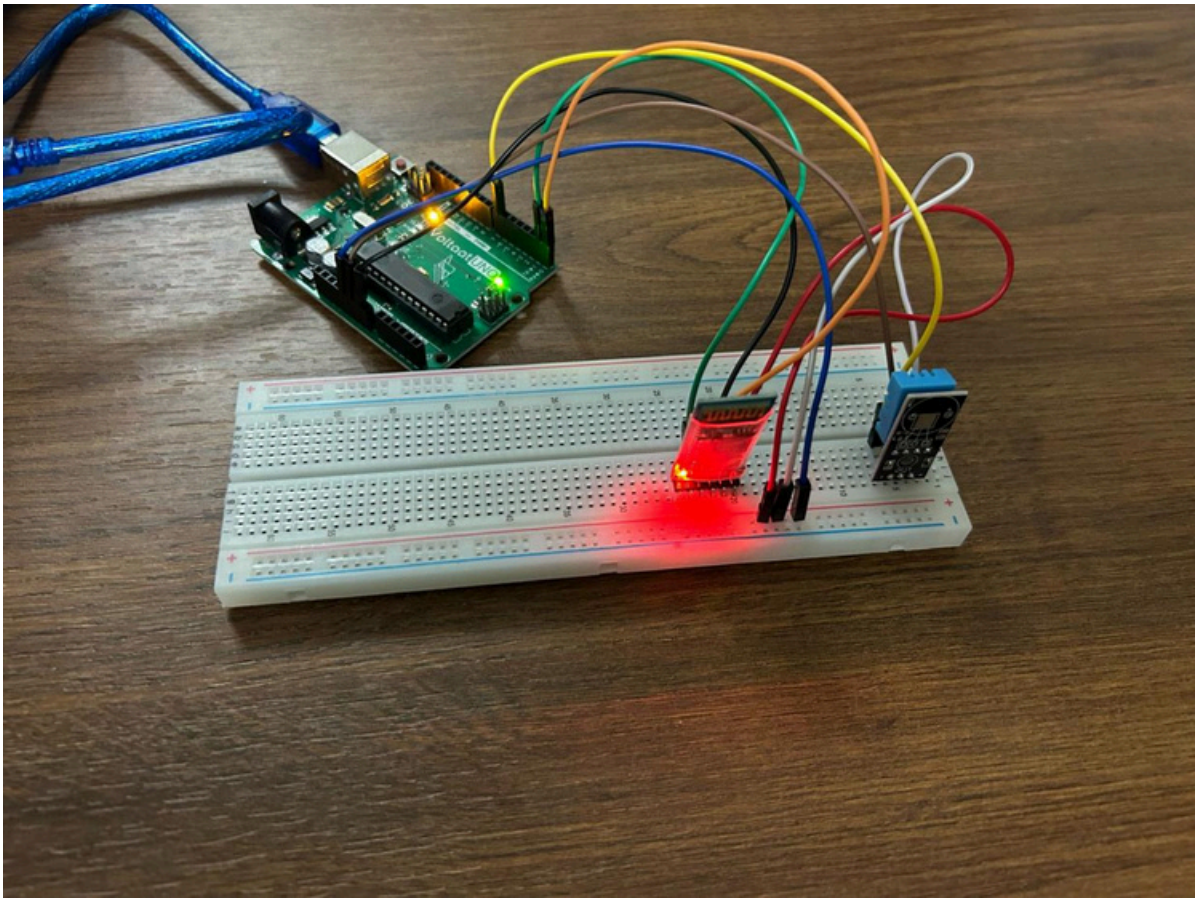


Figure 3: HC-05 Bluetooth Module connected onto Breadboard

Once the circuit was completed, the next step was to program the Arduino toread temperature and humidity from the DHT11 sensor and transfer the data to display it,itwas helpful to look at the website of Arduino (n.d.) to understand how to code in the ArduinoIDE. First, it is a must to include a custom library for communicating with the DHT11 sensor.The line #include <Bonezegei_DHT11.h> imports functions needed to communicate withthe sensor. Without this library, the Arduino wouldn't know how to connect with the sensoror understand the data it sends. The following line, Bonezegei_DHT11 dht(7), declaresan object named dht using the Bonezegei_DHT11 class and specifies that the DHT11 sensor is attached to the Arduino's digital pin 7. This serves as a bridge between the code andthe sensor hardware, with pin 7 being used for reading sensor data.

```
#include <Bonezegei_DHT11.h>

Bonezegei_DHT11 dht(7);
```

Figure 4: Library for DHT11 sensor

Then the setup() function was called, which is done once the Arduino is turned on. Serial.begin(9600) was used in this setup to start serial communication at a baud rate of 9600 bits per second. This enables the Arduino to transmit data to the serial monitor or a device connected the Bluetooth. After that, dht.begin() was used to turn on the DHT11 sensor and prepare it for collecting data from the environment.

```
#include <Bonezegei_DHT11.h>

Bonezegei_DHT11 dht(7);

void setup() {
  Serial.begin(9600);
  dht.begin();
}
```

Figure 5: Code to turn on DHT11 sensor with baud rate

Using dht.getData(), the loop() function first checks whether the sensor hassuccessfully provided the data or not. I got the temperature in Celsius using dht.getTemperature() and saved it in the variable tempC. Then, using the formula (tempC * 9.0 / 5.0)+ 32.0, this number is converted to Fahrenheit and stored in the variable tempF. In addition, the integer variable hum is used to store the humidity, which is obtained using dht.getHumidity(). After that, the code displays every value to the Serial Monitor. The temperature in Celsius is shown with one decimal place followed by "°C", as well as the Fahrenheit version with one decimal place followed by "°F" and the humidity level is shown with the percentage symbol "%". Finally, the line delay(2000) gave two-seconds before the loop resumed again. This delay is important because the DHT11 sensor needs at least 1 to 2 seconds between readings to work properly.

```
#include <Bonezegei_DHT11.h>

Bonezegei_DHT11 dht(7);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  if (dht.getData()) { // get all data from DHT11
    float tempC = dht.getTemperature();       // temperature in Celsius
    float tempF = (tempC * 9.0 / 5.0) + 32.0;  // convert to Fahrenheit
    int hum = dht.getHumidity();              // humidity

    // Print each value on a new line with updated labels
    Serial.print("Temp in Celsius: ");
    Serial.print(tempC, 1);
    Serial.println("°C");

    Serial.print("Temp in Fahrenheit: ");
    Serial.print(tempF, 1);
    Serial.println("°F");

    Serial.print("Humidity: ");
    Serial.print(hum);
    Serial.println("%");
  }

  delay(2000); // delay at least 2 seconds for DHT11 to read the data
}
```

Figure 6: Full code showing how to get the temperature and humidity

After uploading the Arduino code and connecting the hardware, the MIT App Inventorwas used to build a mobile application that could receive data through the HC-05 Bluetooth. At the top of the user interface, it shows “Temperature and Humidity test”. The main interactive component beneath this is a button named "Connect to Bluetooth” which is a ListPicker1 component styled as a button. When clicked, the ListPicker shows a list of available Bluetooth devices, allowing the user to select the correct HC-05 Bluetooth connected to the breadboard. Once a device has been chosen, the app continues to connect to the Bluetooth linking it to the programming blocks that allow the app to work.

The Label1 component is put beneath the button to display real-time sensor readings. This label displays the text "Values will appear here", which informs the user of its purpose as

the app receives data from the Arduino, this label updates and shows both the temperature and humidity. Under this, there is a Button1 called “Disconnect from Bluetooth”, which is there to allow the user to disconnect from the current Bluetooth they are connected to. The components in the background that the user cannot see are Clock1, which serves as a timer that regularly checks for incoming data and BluetoothClient1, which manages all Bluetooth connectivity. The extra labels are there to add my name and student number.

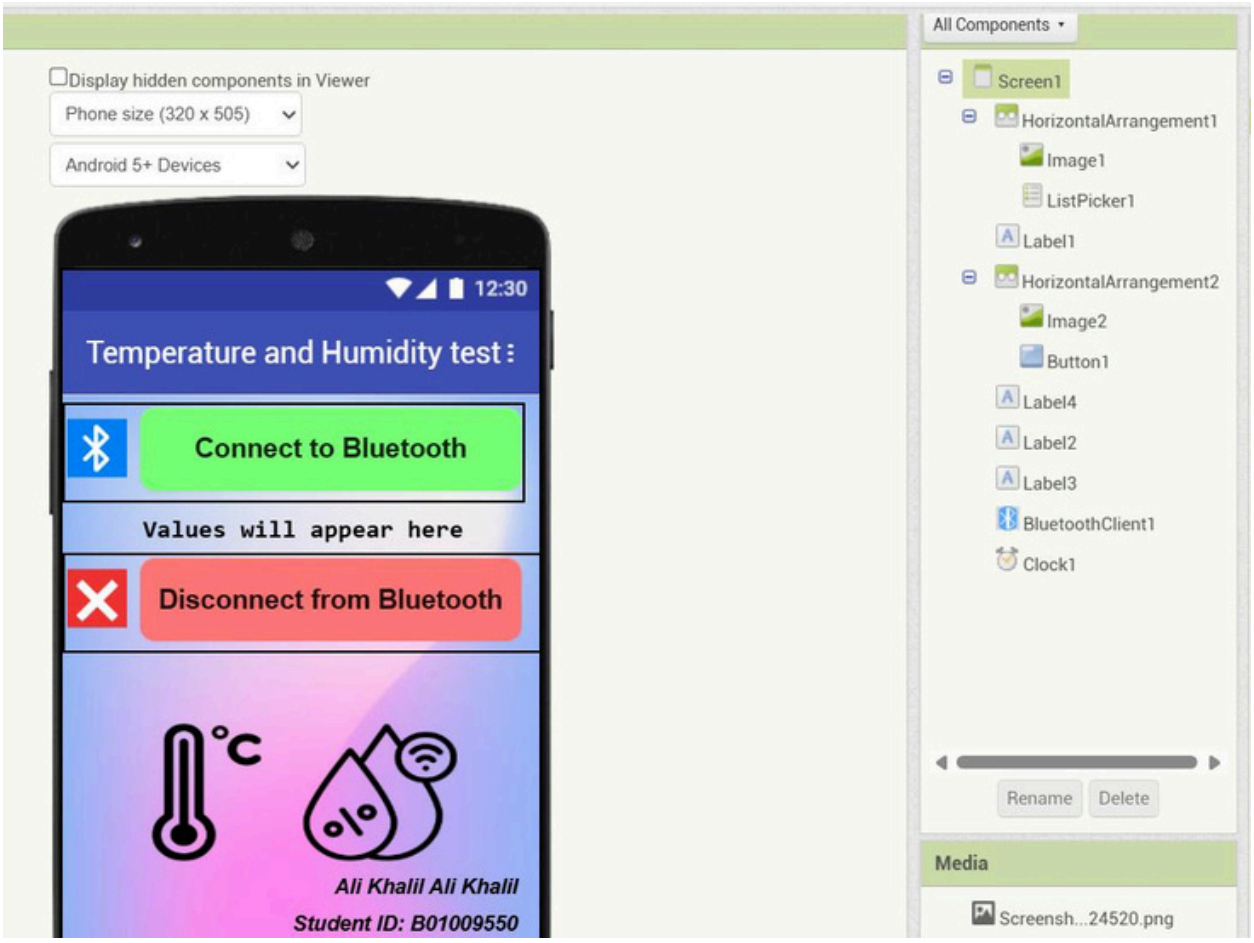


Figure 7: User interface of app including components used

The app's programming blocks were created to support Bluetooth connections and continuous data collection. It was important to understand how to use the MIT blocks by doing some research and I did this by using the (MIT App Inventor, n.d.) website. The first block ListPicker1.BeforePicking sets up the dropdown menu (ListPicker1) to show available Bluetooth devices. It does this by sending the contents of the list to BluetoothClient1.AddressesAndNames, which detects nearby linked Bluetooth devices. When the user selects a device from the list, the ListPicker1.AfterPicking block is activated, the app then connects to the selected device using BluetoothClient1.Connect

with the device address found in ListPicker1.Selection. If the connection is successful, it updates the device list.

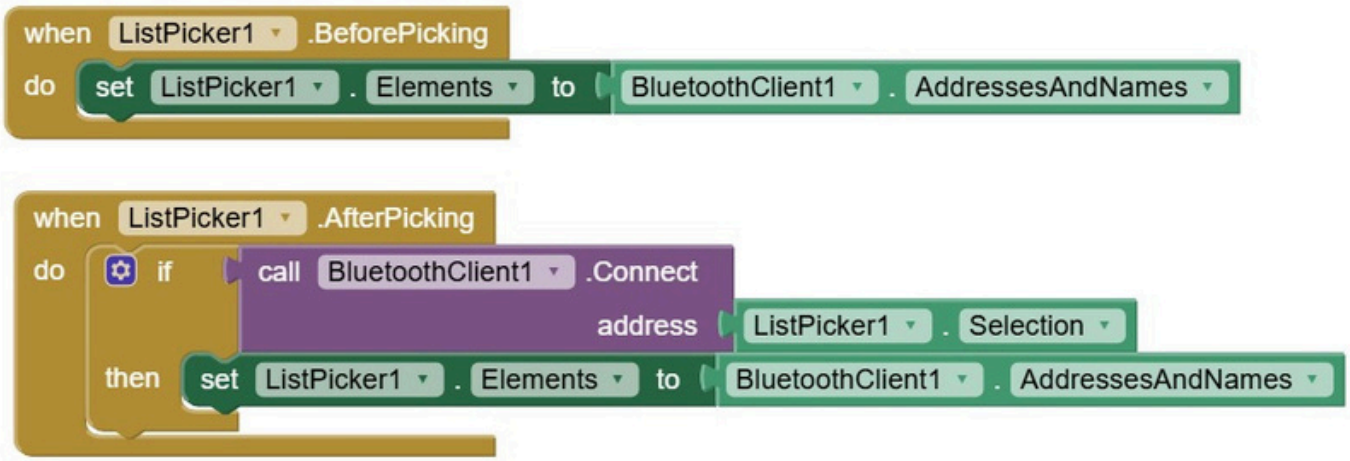


Figure 8: Blocks used to connect to Bluetooth

The third block, Clock1.Timer is required to receive the sensors data from the Arduino. This timer is activated on a regular basis and uses BluetoothClient1.BytesAvailableToReceive > 0 to determine whether there are any bytes available for reading. If data is available, the app will find it by the BluetoothClient1.ReceiveText block. The numberOfBytes is set according to the number of bytes available at the time, ensuring all the data is collected. The data received which is the temperature and humidity values from the Arduino is then displayed as a label on the app's interface using Label1.Text, providing the user to keep track of real-time environmental readings on the Android phone. Lastly, when Button1.Click is needed so that when the button is clicked it calls the BluetoothClient1.Disconnect to disconnect from the Bluetooth it is currently connected to. After finishing the blocks, it was time to build the app and use the QR code to download the app onto the phone.

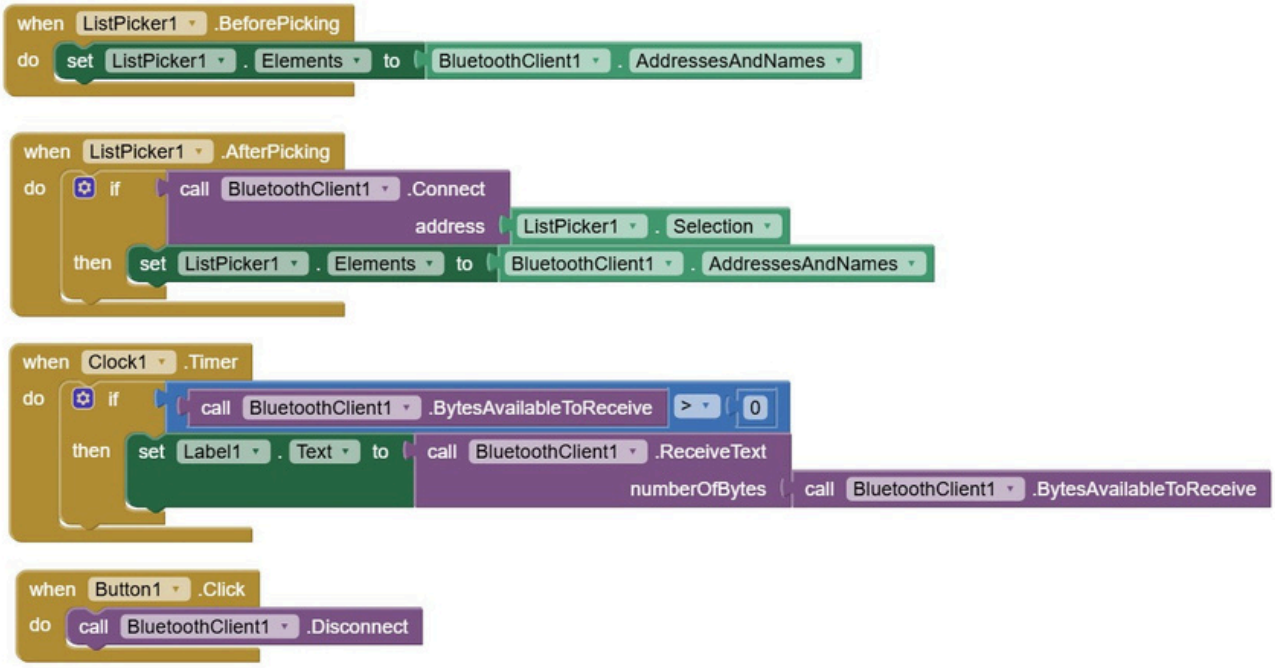


Figure 9: Full blocks used for app to display data and disconnect Bluetooth button

3. Results

After finishing setting up the hardware on the breadboard, I was able to confirm that the HC-05 Bluetooth module got power and was working properly by seeing its blinking red LED. The blinking LED indicated that the Bluetooth was ready to connect to a device, confirming that there is electrical power and ground connection between the Arduino and the HC-05.

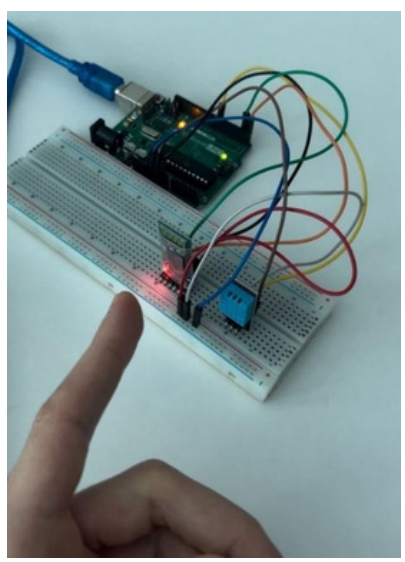
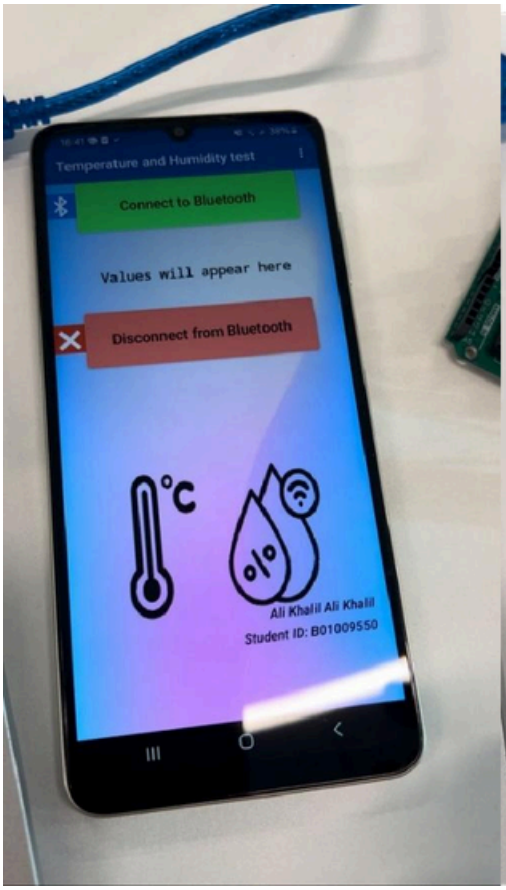
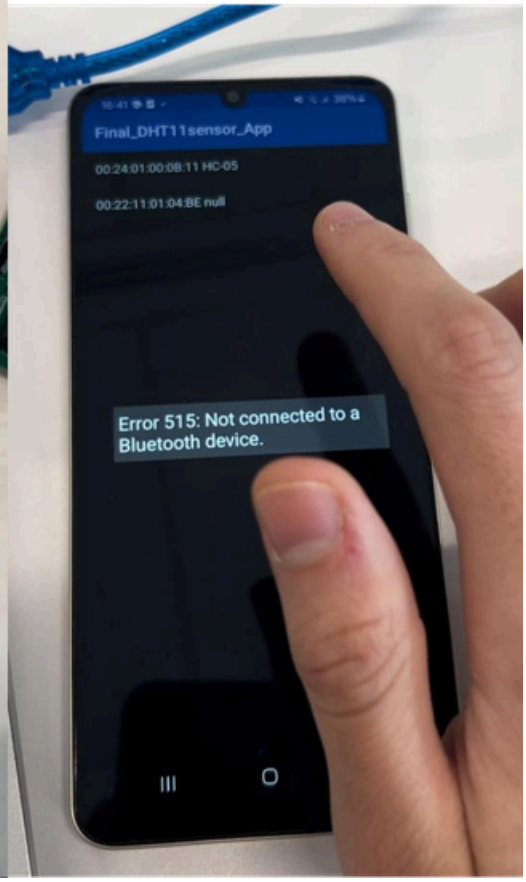


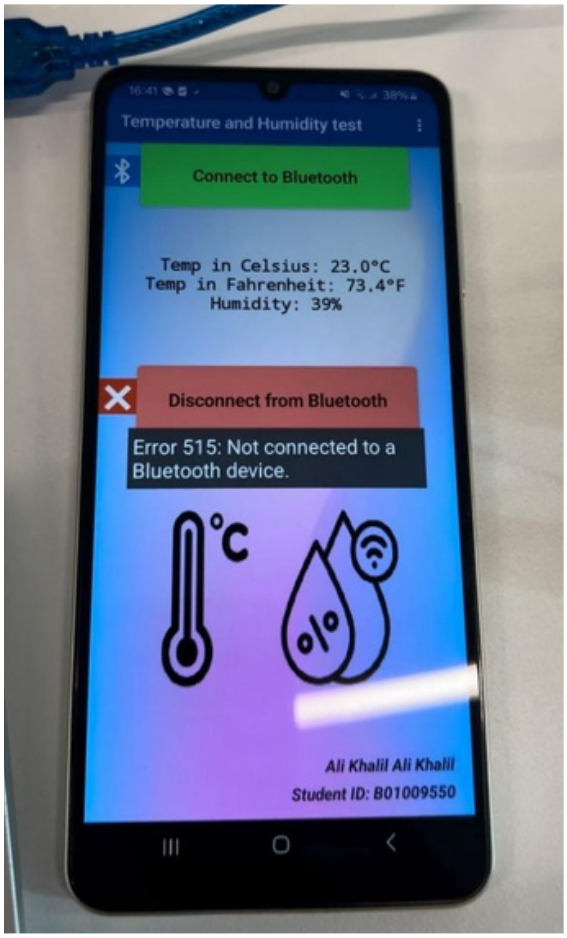
Figure 10: Full working circuit of HC-05 Bluetoothand DHT11 sensor

Ali Khalil Ali Khalil
B01009550
FCE009 Report assignment

After opening the mobile application and clicking the "Connect to Bluetooth" button, a list of available nearby Bluetooth devices appeared. I then chose the right module from the list, and after successfully pairing the Bluetooth, the LED on the HC-05 changed to a slower blinking pattern, which meant that there was a successful Bluetooth connection. I watched the phone interface, which had been created with MIT App Inventor, and was delighted to see the live sensor data show exactly where the label read "Values will appear here." The temperatures showed was 23.0°C and 73.4°F, and the humidity level was 39%, indicating that the DHT11 sensor was working properly and communicating data wirelessly through Bluetooth.







Figures 11, 12, 13: Process of connecting to Bluetooth on App anddisplay of sensor data

These same values were displayed on the Arduino IDE's serial monitor after uploading and running the code, proving data consistency across both platforms. This showed that the Arduino was successfully collecting data from the sensor, formatting it, and communicating it to both the serial monitor and the Android application by Bluetooth. These findings confirm that I have made the appropriate wiring, had the correct logic in the Arduino code and the successful implementation of the MIT app for real-time wireless sensor monitoring.

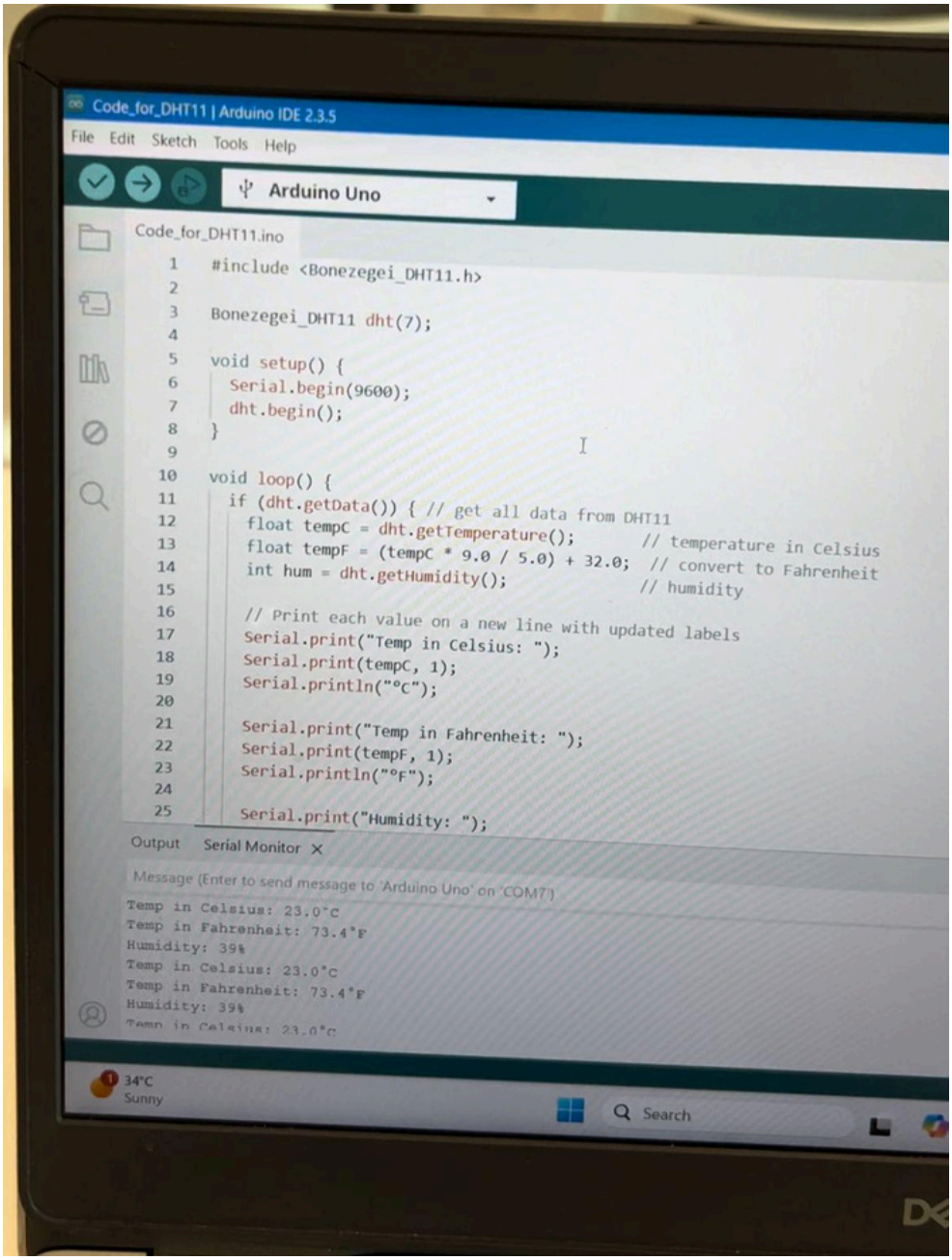


Figure 14: Working code with data displayed in the serial monitor

4. Discussion and Reflection

Working on this Arduino temperature and humidity monitoring system has been an enjoyable and educational experience. Throughout the project, I was responsible for making the circuit, programming the Arduino, installing the HC-05 Bluetooth module, DHT11 sensor, and creating the mobile interface in MIT App Inventor. I began the project with no background knowledge in electronics and mobile app development, but by the end, I had effectively

combined several hardware and software components into a working system that collects, transmits, and displays sensor data in real time. One of my accomplishments was successfully wiring the DHT11 sensor and HC-05 module on the breadboard without any prior experience. I paid extra attention to voltage, ground, and serial pin connections, as wrong wiring might easily harm components or cause failures in communicating the data I needed to be transferred.

I also effectively developed and debugged the Arduino code, which required understanding how to communicate with the DHT11 library and retrieve the sensors data. Another key effort was to create the mobile app interface with MIT App Inventor, I first struggled to get the blocks to work properly, especially when it came to managing Bluetooth connections and ensuring that received data was shown in real time on the app. However, after many trials, errors and research while working in a team effort with classmates, I was able to solve the issue and design a functional app interface that displayed the temperature and humidity once the device was connected. Also, another issue I faced was when the text of the values on the application was changing into unknow signs and values, this is because of the sensor sampling rate limitation which imposed restrictions on the frequency of data updates, using a more advanced sensor (e.g.: DHT22) with higher accuracy and faster sampling rates would have fixed this.
These achievements required not only problem solving but also patience and understanding, particularly when debugging code and ensuring data between the Bluetooth and phone is being transferred. Reflecting on the process, this project significantly improved my knowledge of embedded systems, real-time data transfer, and user interface development. I learnt how to break down a complex system into smaller parts, as well as how to keep going when something doesn't work the first time.

5.References list:

1. Debele, G.M. and Qian, X., 2020, December. Automatic room temperature control system using Arduino Uno R3 and DHT11 sensor. In 2020 17th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP) (pp. 428-432). IEEE. https://doi.org/10.1109/iccwamtip51612.2020.9317307

2. Mori, G.N. and Swaminarayan, P.R., 2020. Measuring IoT security issues and control home lighting system by android application using Arduino Uno and HC-05

bluetooth module. In Data Science and Intelligent Applications: Proceedings of ICDSIA 2020 (pp. 375-382). Singapore: Springer Singapore. https://doi.org/10.1007/978-981-15-4474-3_41

3. Srivastava, D., Kesarwani, A. and Dubey, S., 2018. Measurement of Temperature and Humidity by using Arduino Tool and DHT11. International Research Journal of Engineering and Technology (IRJET), 5(12), pp.876-878. https://www.irjet.net/archives/V5/i12/IRJET-V5I12167.pdf

4. ArduinoDIY. (2015) Measuring temperature with NTC – the Steinhart-Hart formula. [online] Available at: https://arduinodiy.wordpress.com/2015/11/10/measuring-temperature-with-ntc-the-steinhart-hart-formula/ [Accessed 4 April 2025].

5. Arduino. (n.d.) Temperature Sensor with Arduino Nano and Matter. [online] Available at: https://docs.arduino.cc/tutorials/nano-matter/matter-temp-sensor/ [Accessed 5 April 2025].

6. Circuit Basics. (n.d.) How to Set Up the DHT11 Humidity Sensor on an Arduino. [online] Available at: https://www.circuitbasics.com/how-to-set-up-the-dht11-humidity-sensor-on-an-arduino/ [Accessed 8 April 2025].

7. Dejan. (n.d.) Arduino and HC-05 Bluetooth Module Tutorial. [online] HowToMechatronics. Available at: https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/ [Accessed 8 April 2025].

8. MIT App Inventor. (n.d.) Understanding Blocks. [online] Available at: https://appinventor.mit.edu/explore/understanding-blocks [Accessed 12 April 2025].